Sprint 7 - Drupal Development Report

Prepared by: Aicha Nait Dayaa

Supervised by: Mr. Abdelmajid Bendrif
Mr. Hamza Bahlaouane

Date: March 9, 2026

1 Introduction:

This report provides answers to the multiple questions provided in the Drupal development guide. In parallel, we developed two modules - the **Hello World** and **Anytown** modules, to practice and learn at the same time. Below is the Git repository containing these two modules:

https://github.com/a-naitdayaa/drupal_custom_modules

2 Routes, Controllers and Services

2.1. Menus

Q1. How do we control or sort the menus (weight) ?

The order of menu items is controlled using the **weight** property in the YAML file **hello_world.links.menu.yml**.

Menu links are sorted from the **lowest weight to the highest weight**. Therefore, a negative weight makes a menu item appear before others, while higher values move it further down in the menu.

Also, each menu item should specify a **route_name**, which is defined in the file **hello_world.routing.ym** The route connects the menu link to a specific page that is handled by a controller.

```
  hello_world.first_item:
    title: 'First Item'
    route_name: hello_world.first_route
    menu_name: main
    weight: -10

  hello_world.second_item:
    title: 'Second Item'
    route_name: hello_world.second_route
    menu_name: main
    weight: 0
```

In this example, the **First Item** should appear before **Second Item** because it has a lower weight (-10).

Q2. How do i setup child menus ?

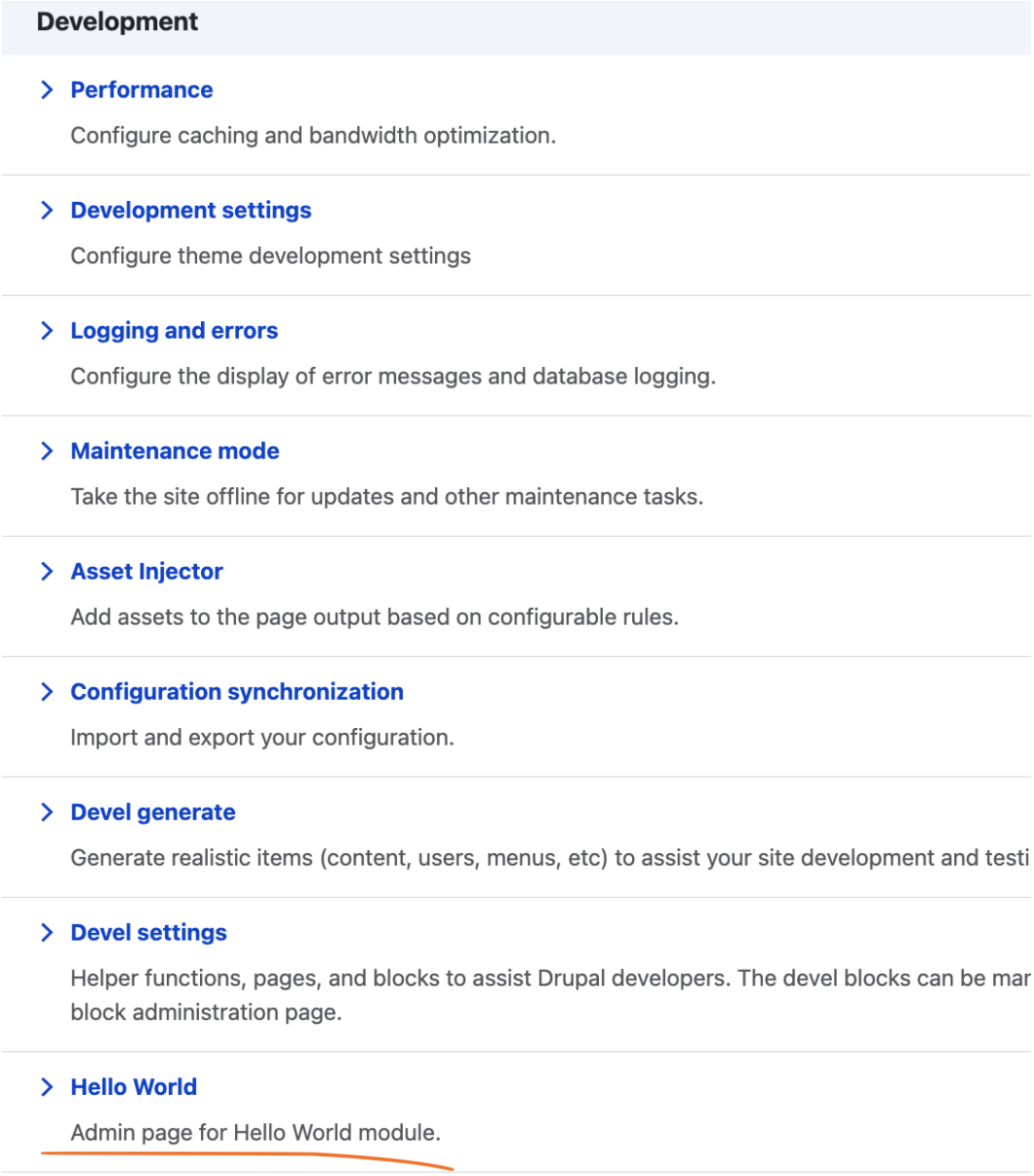
In order to setup child menus, we need to define the **parent** key, in the same YAML file as previously **MODULE.links.menu.yml**

The **parent** key specifies the menu item under which the new link will appear.

```
hello_world.admin:
  title: 'Hello World'
```

```
  description: 'Admin page for Hello World module.'
  route_name: hello_world.content
  parent: system.admin_config_development
  weight: 100
```

In this example, the menu item **Hello World** is added as a child of the menu `system.admin_config_development`.



2.2. Controllers

Q1. How do we retrieve a query string in a Controller ?

In a controller, query string parameters are retrieved from the **Request** object. The controller method can receive this object as an argument and access query parameters through the `query` property.

For example, given the URL:
/example?name=John
the parameter `name` can be retrieved as follows:

```
public function example(Request $request) {
  $name = $request->query->get('name');
  return [
    '#markup' => 'Hello ' . $name,
  ];
}
```

Q2. What is Guzzle ?

- **Guzzle** is a PHP HTTP client library that Drupal uses via the `http_client` service, to send or receive HTTP requests to external APIs or services.

Q3. What is Logger and where do its messages appear? and how to check for them ?

Drupal provides a **Logging API** that uses the service `logger.factory` to create logger channels. These channels allow each module to write messages to Drupal's logging system.

We can write a log message as follows:

```
\Drupal::logger('hello_world')->info('This is a log message from the controller.');
```

In this example, `hello_world` is the logging channel. We can use different severity levels, such as `debug`, `info`, `notice`, `warning`, and `error`.

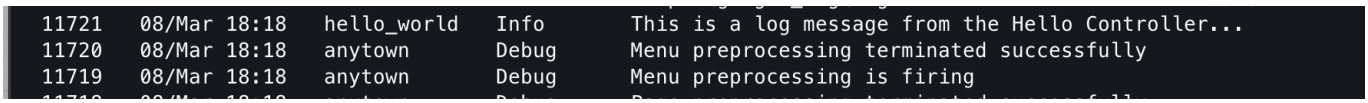
By default, Drupal stores log messages using the **Database Logging module** (`dblog`), which saves logs in the database.

Logs can be viewed in two ways:

- **Drupal administration interface**: Navigate to **Reports** → **Recent log messages**.

- **Command line using Drush**:

```
./vendor/bin/drush ws
```



Q4. How do you send JSON response in a Controller instead of a display ?

To send a JSON response in a drupal's controller instead of a render array, all we need is to return a JSonResponse object, and drupal will correctly set the `Content-Type: application/json` header.

```
class MyApiController extends ControllerBase {
  public function getData() {
    $data = [
      'status'  => 'success',
      'message' => 'Here is your data',
      'items'   => [1, 2, 3],
    ];
    return new JsonResponse($data, 200);
  }
}
```

2.3. Services

Q1. Where can i find the services defined by Drupal core ?

The YAML file that defines all the drupal core services, is placed in the file **/web/core/core.services.yml**. It contains the definitions of the services, including their class, arguments, and dependencies.

- Listing all services

To list all services, devel provides the following command, which lists core services, contributed module services and also custom module services.

```
drush devel:services
```

Q2. How do you inject other needed services into your service ?

Services are injected using dependency injection through the service constructor. The process can be separated into 3 parts :

- We declare the needed dependencies in the YAML file **hello_world.services.yml** of the module :

  we list the services our service needs under the `arguments` key.

```
services:
  hello_hello.my_service:
    class: Drupal\hello_world\Hello
    arguments: ['@logger.factory', '@entity_type.manager']
```

- Receiving dependencies in the service constructor : In our service class, we define a constructor that accepts the required services. We type-hint each argument with the corresponding interface, so that Drupal's service container knows what type each dependency is, and so that our code is decoupled from specific implementations.

```
        public function __construct(LoggerInterface $logger) {
          $this->logger = $logger;
        }
```

- And lastly, we are then able to use the required injected service in our service class.

2.4. Themes

Q1. How do you return a twig template in a Controller ?

Instead of returning a simple **#markup** render array, a controller can return a **#theme** render array that maps to a Twig template. To use a custom template, it must first be registered in the module via the hook **hook_theme()** in the **hello_world.module** file.

1. Returning the twig template from the controller

```
        public function content(){
            return [
              '#theme' => 'hello_block',
              '#custom_data' => ['age' => '22', 'DOB' => '21 Juin
                 2003'],
              '#custom_string' => 'Hello Everyone, welcome to the
                 hello_world page!',
            ];
          }
```

2. Registering the template in the module

```
        function hello_world_theme() {
          return [
            'hello_block' => [
              'variables' => [
              'custom_data' => [],
              'custom_string' => '',
              ],
              'template' => 'hello-block',
            ],
          ];
        }
```

3. Defining the custom twig template file

   The corresponding Twig template file should be placed in the module's **templates/hello_world.html.** folder.

Q2. How do you add an external JavaScript library to a Controller?

Libraries are declared in the file **module.libraries.yml**, defined in the module's folder, then attached to the render array using the **#attached** render array property.

First then, We need to define the external js file in the YAML file anytown.libraries.yml:

```
tailwind:
  js:
    https://unpkg.com/@tailwindcss/browser@4:
      { type: external }
```

Then we'll attach it to the renderrable array returned by the controller through the build method:

```
public function build() {
  return [
    '#attached' => [
      'library' => [
        'anytown/tailwind',
      ],
    ],
  ];
}
```

Q3. Passing data from Drupal module to javascript

To be able to pass data from a Drupal module to javascript, we first need to create the library file that references the asset files (js, css, etc.) and that lists all assest's dependencies in the following YAML file: **Module_name.libraries.yml**.

Second, there are many ways to attach the assets to the page:

- Through a render array
- Through a twig template
- Through the hook **hook_page_attachments()**

Finally, to be able to pass data to javascript, we need to expose it through a DrupalSettings object.

2.5. Translation

Q1. Translation - search keyword for `$this->t()`

```
    $this->t('Hello, @name!', ['@name' => $username])
```

The string that appears in the translation interface at `/admin/config/regional/translate` is the **source string**, with placeholders :

Hello, @name!

So the search keyword to use in the translation UI filter would be: `Hello, @name!`.

Q2. How do you make a string translatable in Drupal JavaScript files?

In Drupal JavaScript, translatable strings are wrapped with the `Drupal.t()` function. This function is the JavaScript equivalent of `$this->t()` in PHP that allows Drupal's translation system to detect and replace interface strings. (e.g.)

```
(function (Drupal) {
  'use strict';
  Drupal.behaviors.anytown = {
    attach: function (context, settings) {
        .
        .
      toggleButton.textContent = Drupal.t('Toggle extended forecast');
        .
        .
    }
  };

})(Drupal);
```

3 Hooks

Hooks are functions or methods that Drupal calls at specific points during its execution, allowing modules to react to or modify what is happening without touching the original code. For example, when Drupal builds a form, it calls all implementations of `hook_form_alter`, giving every module a chance to modify that form before it is rendered.

There are two main approaches of defining hooks:

Q1. Procedural approach

A function named `MODULENAME_hookname()` in the module's `.module` file:

```
/**
* Implements hook_theme().
*/
function hello_world_theme() {
  return [
    'hello_block' => [
      'variables' => [
      'custom_data' => [],
      'custom_string' => '',
      ],
      'template' => 'hello-block',
    ],
  ];
}
```

Q2. OOP approach

Using the `#[Hook]` PHP attribute in a dedicated class under `src/Hook/`:

```
class AnytownHelp {
  /**
   * Current user service.
   *
   * @var \Drupal\Core\Session\AccountProxyInterface
   */
  private $currentUser;

  public function __construct(AccountProxyInterface $current_user) {
    $this->currentUser = $current_user;
  }

  /**
   * Implements hook_help().
   */
```

```
  #[Hook('help')]
  public function help($route_name, RouteMatchInterface $route_match) {
    if ($route_name === 'help.page.anytown') {
      $name = $this->currentUser->getDisplayName();
      return '<p>' . $this->t("Hi %name, the anytown module provides code specific to the Anytown Farmer's market website. This includes the weather forecast page, block, and related settings.", ['%name' => $name]) . '</p>';
    }
  }
}
```

Key difference: The OOP approach allows proper dependency injection into the hook class constructor, making it more testable and decoupled.

Q3. Hooks Provided by the Metatag Module

The hooks provided by a module, are defined in the file **module.api.yml**. For the Metatag module : **modules/contrib/metatag/metatag.api.php**

Common hooks include:

- **hook_metatag_alter()** : Allows other modules to modify existing metatags before they are rendered.

```
 * Alter the meta tags for pages that are not of content entities.
 *
 * @param array $metatags
 *   The special meta tags to be added to the page.
 * @param array $context
 *   The context for the current meta tags being generated. Will contain the
 *   following:
 *   'entity' - The entity being processed; passed by reference.
 */
no usages
function hook_metatags_alter(array &$metatags, array &$context) {
  // Exclude meta tags on frontpage.
  if (\Drupal::service( id: 'path.matcher')->isFrontPage()) {
    $metatags = NULL;
  }
}
```

- **hook_metatags_attachments_alter** : Allowing Alteration of the final attachments array before output

```php
/**
 * Alter the meta tags for any page prior to page attachment.
 *
 * @param array $metatag_attachments
 *   An array of metatag objects to be attached to the current page.
 */
no usages
function hook_metatags_attachments_alter(array &$metatag_attachments) {
  if (\Drupal::service( id: 'path.matcher')->isFrontPage() && \Drupal::currentUser()->isAnonymous()) {
    foreach ($metatag_attachments['#attached']['html_head'] as $id => $attachment) {
      if ($attachment[1] == 'title') {
        $metatag_attachments['#attached']['html_head'][$id][0]['#attributes']['content'] = 'Front Page Title for Anonymous
      }
  }
  }
}

/**
```

while the file **module_name.module** contains the actual implementation and logic used by the module.

Q4. Altering Forms with `hook_form_alter`

The hook responsible for altering forms is `hook_form_alter`. It fires every time Drupal builds any form, giving modules a chance to add, remove, or modify form elements.

It receives three arguments:

Argument	Description
`&$form`	The full form render array, passed by reference
`$form_state`	A `FormStateInterface` object holding current form state and submitted values
`$form_id`	The uniqu ID of the form being built

To extend a form, we simply add a new element to the `$form` array using a render array with the desired `#type`, in this case `textfield`.

```php
function mymodule_form_alter(&$form, &$form_state, $form_id) {
  if ($form_id === 'user_login_form') {
    $form['extra_note'] = [
      '#type' => 'textfield',
      '#title' => t('Extra Note'),
    ];
  }
}
```

Q5. Passing `is_front` to Twig via `hook_preprocess_page`

Using the `isFrontPage()` method from the **path.matcher** service, We checked whether the current page is the front page or not, and passed that value as a Twig variable named `is_front`.

Implementation

```
class AnytownPreprocessHooks {
  protected $pathMatcher;
  public function __construct(
    protected PathMatcherInterface $pathMatcher,
  ) {
    $this->pathMatcher = $path_matcher;
  }

  #[Hook('preprocess_page')]
  public function preprocessPage(&$variables): void {
    $variables['is_front'] = $this->pathMatcher->isFrontPage();
  }
}
```

And then we use the variable **is_front** in a template to verify it works :

```
{{ dump(is_front) }}

{% if is_front %}
<h1>This is the front page!</h1>
{% else %}
<h1>Not the front page.</h1>
{% endif %}
```

Visiting a page other than the homepage would output the message **Not the front page.**

Q6. Adding a Viewport Metatag via `hook_page_attachments_alter`

The hook `hook_page_attachments_alter` fires after all modules have had a chance to attach assets and HTML head elements to the page. It allows altering or adding to the final `#attached` render array before the page is sent to the browser.

```
#[Hook('page_attachments_alter')]
```

```
public function pageAttachmentsAlter(array &$attachments): void {
  $attachments['#attached']['html_head'][] = [
    [
      '#type' => 'html_tag',
      '#tag' => 'meta',
      '#attributes' => [
        'name' => 'viewport',
        'content' => 'width=device-width, initial-scale=1, shrink-to-fit=no',
      ],
    ],
    'viewport',
  ];
}
```

This produces the following HTML in the <head> of every page:

```
<meta name="HandheldFriendly" content="true">
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
<style>…</style>
<link rel="icon" href="/core/themes/olivero/favicon.ico" type="image/vnd.microsoft.icon">
```

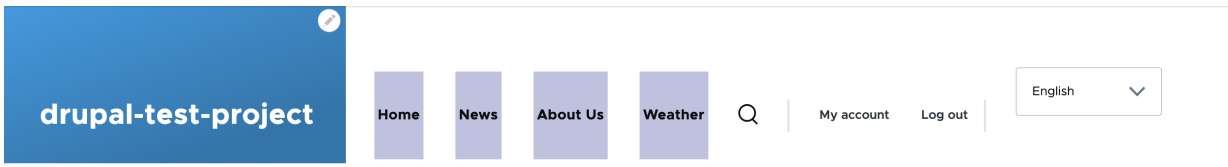
Q7. Adding a CSS Class to Menu Items via `hook_preprocess_menu`

The hook `hook_preprocess_menu` fires before any menu is rendered, giving access to the menu items tree. It is the correct place to add custom CSS classes, attributes, or modify attributes before Twig renders the menu.

Implementation

```
#[Hook('preprocess_menu')]
public function anytown_preprocess_menu(array &$variables) : void {
  if($variables['menu_name'] === 'main'){
    foreach($variables['items'] as &$item){
      $item['attributes']->addClass('my-custom-class');
    }
  }
  $variables['#attached']['library'][] = 'anytown/menu';
}
```

And before implementing this hook, we for sure need to define the css file and register it in the **anytown/libraries.yml** file for it to be recognized by Drupal.



Q8. Replacing the Site Logo via `hook_preprocess_block`

The hook `hook_preprocess_block` fires before any block is rendered. By checking `$variables['plugin_id'`
we can target a specific block and modify its template variables before Twig renders it.

Implementation

```
#[Hook('preprocess_block')]
public function anytown_preprocess_block(array &$variables) {
  if($variables['plugin_id'] === 'system_branding_block'){
    $variables['content']['site_logo'] = [
      '#theme' => 'image',
      '#uri' => 'https://static.cdnlogo.com/logos/d/88/drupal-wordmark.
        svg',
      '#alt' => 'Drupal Logo',
      '#access'  => TRUE,
    ];
  }
  $variables['#attached']['library'][] = 'anytown/branding';
}
```

```
array:3 [▼
 "site_logo" => array:4 [▼
   "#theme" => "image"
   "#uri" => "https://static.cdnlogo.com/logos/d/88/drupal-wordmark.svg"
   "#alt" => Drupal\Core\StringTranslation\TranslatableMarkup {#9542 ▶}
   "#access" => true
 ]
 "site_name" => array:2 [▶]
 "site_slogan" => array:2 [▶]
]
```

4 Plugins & Forms

4.1. Form Validation

Q1. Where can you validate form data?

The form validation happens in two places.

- The primary place is the `validateForm()` method inside the form class. This method receives `$form` and `$form_state` as arguments.

```
public function validateForm(array &$form, FormStateInterface $form_state) {
  $name = $form_state->getValue('name');

  if (strlen($name) < 3) {
    $form_state->setErrorByName(
      'name',
      $this->t('The name must be at least 3 characters long.')
    );
  }
}
```

- The second place is through the hook `hook_form_alter`, where we can push a custom validation callback into the `$form['#validate']` array.

```
#[Hook('form_user_register_form_alter')]
  public function FormUserRegistrationAlter(array &$form, FormStateInterface $form_state) : void {
    $form['#validate'][] = [self::class, 'validateFormUserRegistrationForm'];
    }

    /**
   * Custom validation handler for the user registration form.
   */
  public static function validateFormUserRegistrationForm(&$form, FormStateInterface $form_state)
  {
    if ($form_state->getValue('mail') === 'anytown@example.com') {
      $form_state->setErrorByName('name', t('The username "anytown" is invalid. Please choose a different name.'));
    }
  }
```

4.2. Rendering a Form Inside a Block

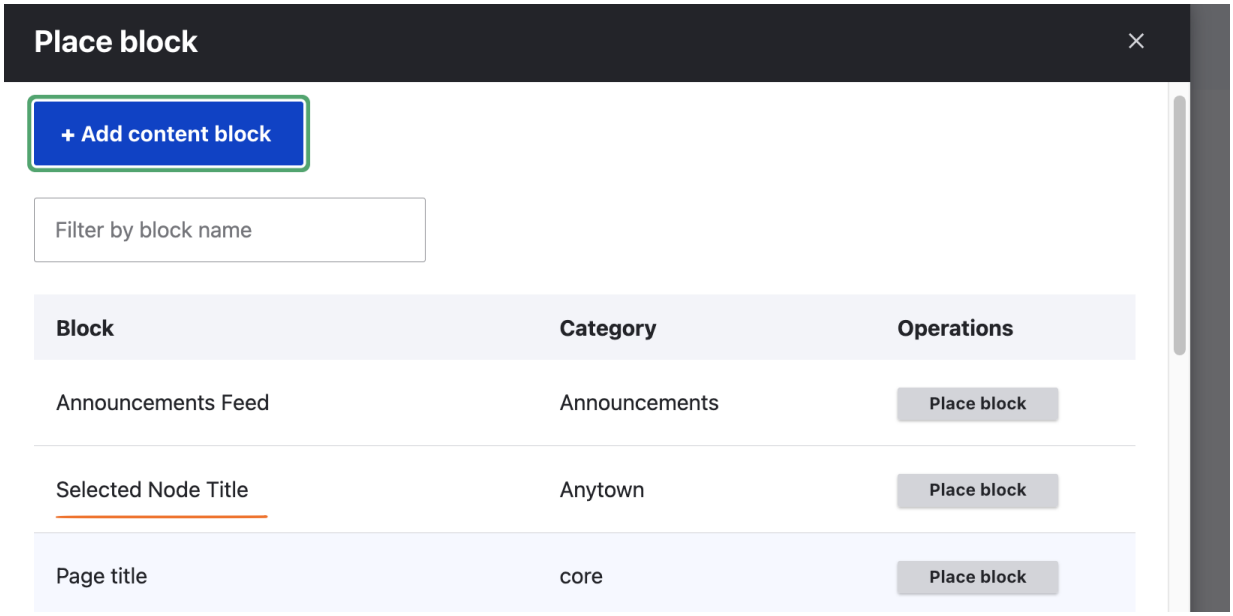
Q1. How to embed a Form in a Block's build method

We can build forms anywhere using the **form_builder** service.

Inside the **build()** method of a Block plugin, we use **\Drupal::formBuilder()→getForm()** and pass the class name of the form we want to render. Drupal will instantiate the form, run it through the Form API lifecycle.

```
#[Block(
  id: "anytown_selected_node_block",
  admin_label: new TranslatableMarkup("Selected Node Title")
)]
class SelectedNodeBlock extends BlockBase implements
   ContainerFactoryPluginInterface{

  protected $entityTypeManager;

  public function __construct(array $configuration, $plugin_id,
     $plugin_definition, EntityTypeManagerInterface $entityTypeManager) {
    parent::__construct($configuration, $plugin_id, $plugin_definition);
    $this->entityTypeManager = $entityTypeManager;
  }

  public static function create(ContainerInterface $container, array
     $configuration, $plugin_id, $plugin_definition) {
    return new static(
      $configuration,
      $plugin_id,
      $plugin_definition,
      $container->get('entity_type.manager')
    );
  }

  public function build() {
    $nid = \Drupal::state()->get('anytown.selected_node');

    if (!$nid) {
      return [
        '#markup' => 'No node selected yet.'
      ];
    }

    $node = $this->entityTypeManager
      ->getStorage('node')
```

```
      ->load($nid);

    if (!$node) {
      return [
        '#markup' => 'Node not found.'
      ];
    }

    return [
      '#markup' => $node->getTitle(),
    ];
  }
}
```



4.3. Redirecting and Displaying Messages After Form Submission

Q1. How to redirect a user after submitting a form

Redirects after form submission are handled inside the `submitForm()` method using `$form_state→setRed` which accepts a route name, or `$form_state→setRedirectUrl()` which accepts a `Url` object.

```
public function submitForm(array &$form, FormStateInterface $form_state) {
  // Redirecting using a named route defined in *.routing.yml
  $form_state->setRedirect('anytown.some_route');

  // Or redirecting using a Url object
  $form_state->setRedirectUrl(Url::fromRoute('anytown.some_route'));

```

```
  // Or
  $form_state->setRedirectUrl(Url::fromUri('https://example.com'));
}
```

Q2. How to display a status message after submission

We can display status messages using the `messenger` service. Messages can have different types:

- Status : green success messages
- Warning : yellow/orange caution messages
- Error : red error messages

Example :

```
public function submitForm(array &$form, FormStateInterface $form_state) {
  $this->messenger()->addStatus(
    $this->t('Your form was submitted successfully.')
  );

  $form_state->setRedirect('anytown.some_route');
}
```

4.4. Controlling Field Visibility with `#access`

Q1. How to hide a field for anonymous users

We can hide almost any field using the **#access** property, including form fields. To hide a field from anonymous users, we only need to set the **#access** property to authenticated users only :

```
$form['secret_field'] = [
  '#type' => 'textfield',
  '#title' => $this->t('Secret Field'),
  '#access' => \Drupal::currentUser()->isAuthenticated(),
];
```

There are many ways to use **#access**, such as hiding fields for certain roles, permissions, or other conditions. However, the use of the property **#access** alone does not provide security, because it only prevents the field from being rendered.

4.5. Grouping Fields in a Form

Q1. How to group fields like the field_group module

To group fields in a form, we can use one of the 2 element types **details**, or **fieldset**.

The difference between the two is that **details** renders as a collapsible HTML element, while **fieldset** is a static bordered group. Adding `'#open' => TRUE` to a **details** element makes it expanded by default.

Example :

```
$form['personal_info'] = [
  '#type' => 'fieldset',
  '#title' => $this->t('Personal Information'),
];

$form['personal_info']['first_name'] = [
  '#type' => 'textfield',
  '#title' => $this->t('First Name'),
];

$form['personal_info']['last_name'] = [
  '#type' => 'textfield',
  '#title' => $this->t('Last Name'),
];
```

5 Data Types

5.1. Configuration API

Q1. How to inspect active configuration using drush php

To inspect the active configuration of the `eu_cookie_compliance` module, we can use `drush php` to open an interactive PHP shell, then use the `config` service to retrieve and dump the settings.

Inside the shell, we retrieve the config object and dump all its values:

```
dump(\Drupal::config('eu_cookie_compliance.settings'));
```

The string passed to `\Drupal::config()` is the **config object name**. Calling `->get()` with no arguments returns all values stored under that config object.

Q2. Where is the config file located?

There are two places to look for configuration related to a module.

- The **default configuration** shipped with the module is inside the module's source code at:

`modules/contrib/eu_cookie_compliance/config/install/eu_cookie_compliance.settings.yml`

This file is imported into the database when the module is first installed. It defines the default values for all settings.

- The **active configuration** lives in the database. When we export configuration using `drush config:export`, Drupal writes the current active config to the sync directory :

`config/sync/eu_cookie_compliance.settings.yml`

Q3. Why do we need a `schema.yml` file?

The schema file tells defines the **structure and data types** of a module's configuration. It is located at:

`modules/custom/mymodule/config/schema/mymodule.schema.yml`

Without a schema file, Drupal wouldn't know whether a config value is a string, boolean, integer, or a mapping.

```
anytown.settings:
  type: config_object
  label: 'Anytown settings'
  mapping:
    display_forecast:
      type: boolean
      label: 'Toggle forecast display on/off'
```

```
    location:
      type: string
      label: 'ZIP code where the market takes place'
    weather_closures:
      type: string
      label: 'Weather related closures'
```

5.2. Entity API

Q1. How to load a node

Nodes are loaded through the entity type manager service using the `node` storage handler.

```
    $node = \Drupal::entityTypeManager()
      ->getStorage('node')
      ->load(1);
```

Q2. How to get field values

Once the node is loaded, we can access its field values using the `get()` method or dedicated getter or helper methods:

```
    // Using dedicated getters for base fields
    $title = $node->getTitle();
    $created = $node->getCreatedTime();

    // Using get() for any field
    $body   = $node->get('body')->value;
    $summary = $node->get('body')->summary;
    $custom = $node->get('field_custom')->value;
```

Q3. How to update a field and save the node

To update a field value we use `set()`, then call `save()` to persist the change to the database:

```
    $node->set('field_custom', 'new value');
    $node->set('title', 'Updated Title');
    $node->save();
```

Q4. What is a Constraint?

A Constraint is a validation rule attached to an entity field or an entire entity, and they define what valid data looks like; for example, ensuring a field is not empty, or a username is unique across the system. They are applied during entity validation.

There are many constraints provided by Drupal, such as `NotNull`, `Length`, and `UniqueField`, and we can also define custom constraints.

Q5. What is a View Builder?

A **View Builder** is the service responsible for rendering an entity into a render array. Each entity type has its own view builder, and it can render entities in different display modes.

```
$view_builder = \Drupal::entityTypeManager()
  ->getViewBuilder('node');

// Render in teaser display mode
$build = $view_builder->view($node, 'teaser');

// Render in full display mode
$build = $view_builder->view($node, 'full');
```

Q6. The Role of `accessCheck` in Entity Queries

The property `accessCheck()` controls whether the entity query filters its results based on the current user's permissions.

Passing `TRUE` tells Drupal to only return entities the current user has permission to view, while `FALSE` bypasses all access checks and returns every matching entity regardless of permissions.

```
$nids = \Drupal::entityTypeManager()
  ->getStorage('node')
  ->getQuery()
  ->condition('type', 'article')
  ->condition('status', 1)
  ->accessCheck(TRUE)
  ->execute();
```

Q7. How to retrieve a French translation of a node

Node translations are accessed through the `getTranslation()` method, which accepts a language code and returns the translated version of the entity.

```
$node = \Drupal::entityTypeManager()
  ->getStorage('node')
  ->load(1);

if ($node->hasTranslation('fr')) {
  $translated = $node->getTranslation('fr');
  $title = $translated->getTitle();
  $body  = $translated->get('body')->value;
}
```